

需求文档化、验证与冲突检测

软件工程与计算 II · 模块二 · 单元 5

需求文档是团队协作的基石，验证是质量的最后防线

课程	软件工程与计算 II
专业	软件工程
主题	文档化 · 需求验证 · AI辅助冲突检测

今日学习路径

① 文档化 ② 核心文档 ③ AI辅助SRS ④ 需求管理 ⑤ 验证与冲突 ⑥ Spec驱动开发

为什么? 用例/SRS LLM生成能力 变更/追踪/属性 歧义/冲突检测 工业界SDD实践



四个核心问题，今天结束时你应该能回答：

1. 用例文档和SRS有什么区别？各自适用场景？
2. LLM能自动生成SRS吗？质量如何保障？
3. AI在需求歧义检测和冲突识别方面表现如何？
4. 什么是 Spec-Driven Development？它与传统需求工程有什么关系？

PART 1 · 为什么文档化需求

Why Documents? [SWEBOK 4.1]

Teamwork and Communication

文档的读者远不止开发人员：

角色	关注点
Project Manager	范围、进度、优先级
Architect	系统约束、质量属性
Software Designer	功能细节、接口定义
Programmer	实现逻辑、边界条件
Tester	验收标准、测试用例
Maintainer	变更影响、追溯性

需求规格说明： 在系统用户之间交流需求信息

要求：**简洁、精确、一致和易于理解**

Tips of Documents [SWEBOK 4.1]

Why Standard?

- 统一格式降低沟通成本
- 新成员可以快速上手阅读
- 不同团队之间可以互相审查

Can we solve communication problem with other methods?

- 会议记录会遗忘，口头承诺会变化
- 文档是持久化的沟通载体
- 文档可追溯、可审计、可版本化

文档不是目的，而是确保信息不丢失的手段

需求工程师在文档化阶段的重要工作 [SWEBOK 4.1]

1. 定制文档模板

- 根据项目类型选择合适的文档标准（IEEE 830、Volere 等）
- 根据团队规模和项目规模裁剪模板
- 确定文档的详细程度

2. 编写文档

- 将需求获取和分析的成果转化为规范文档
- 确保每条需求**可追溯、可验证、无歧义**
- 维护需求之间的**交叉引用**关系

PART 2 · 两种核心文档

用例文档 (Use Case Document) [SWEBOK 4.2]

在用户的角度以用例文本为主描述软件系统与外界的交互

基本职责： 把问题域信息和需求传达给软件系统解决方案的设计者

文档结构

部分	内容
一、文档信息	标题、作者、更新历史、写作目的、组织结构
二、用例图/用例列表	系统边界、参与者、用例关系总览
三、用例描述	用例1、用例2、...、用例n 的详细描述

用例文档关注的是"用户和系统之间发生了什么"

软件规格说明文档（SRS）[SWEBOK 4.1]

在软件产品的角度以系统级需求列表的方式描述软件系统解决方案

用例 vs 系统规格对比

维度	用例文档	SRS
视角	用户视角	系统视角
侧重点	交互流程	独立需求
基础	以一次交互为基础	以一次交互中的软件系统处理细节为基础
组织方式	按用例组织	按功能模块/特性组织
适用场景	需求沟通、用户确认	设计、开发、测试

SRS关注的是"系统应该做什么"，每条需求都是独立可验证的

电梯系统SRS片段示例 [SWEBOK 4.1]

刺激/响应序列 (IEEE 830 格式)

#	刺激	系统响应	约束
SR-01	用户在某楼层按上行键	系统记录该楼层的上行请求；电梯到达后开门，按钮灯熄灭	若电梯已在该楼层且空闲，立即开门
SR-02	用户在某楼层按下行键	同上，方向为下行	一楼无下行键，顶楼无上行键
SR-03	用户进入轿厢	红外感应检测到进入，重置关门计时器	感应区被持续遮挡时门保持打开
SR-04	用户在轿厢内按楼层键	该楼层按钮灯亮起；系统将目标楼层加入调度队列	当前楼层按钮按下无效
SR-05	关门计时器到期（5秒）	系统关闭轿门，按调度算法运行至下一目标楼层	门关闭过程中检测到障碍物则重新打开

SRS 以刺激/响应精确定义系统行为，每条需求独立可验证、可追溯

电梯系统SRS片段示例（续）[SWEBOK 4.1]

功能需求

需求 ID	描述	关联刺激
FR-001	系统应在用户按下外部呼叫按钮后 200ms 内 记录请求并点亮按钮灯	SR-01, SR-02
FR-002	系统应在轿厢到达目标楼层后 1 秒内 打开轿门	SR-01, SR-02
FR-003	系统应在无人进出且 5 秒无操作 后自动关闭轿门	SR-03, SR-05
FR-004	系统应在关门过程中检测到 障碍物 时立即停止关门并重新打开	SR-05
FR-005	系统应在轿厢内楼层键按下后 200ms 内 将目标楼层加入调度队列	SR-04
FR-006	系统应在每层楼的 显示面板 实时显示当前楼层和运行方向	—
FR-007	系统应在电梯 满载 时忽略外部呼叫请求，仅响应轿厢内请求	SR-01, SR-02

非功能需求（节选）

需求 ID	类型	描述
NFR-001	性能	从最高层到最低层的运行时间不超过 60 秒
NFR-002	安全	轿门未完全关闭时电梯 禁止运行
NFR-003	可用性	系统年可用率 $\geq$ 99.95% （年停机不超过 4.4 小时）

文档化需求的注意事项 [SWEBOK 4.1]

需求规格说明文档至少要满足：

1. 正确性 (Correctness)

文档内每条需求都**正确、准确**地反映了用户的意图

2. 完整性与一致性 (Completeness & Consistency)

文档记录的需求集在整体上具有**完整性和一致性**

3. 可读性与可修改性 (Readability & Modifiability)

文档的组织方式和需求的书写方式具有**可读性和可修改性**

这三条标准也是后续需求验证的核心检查项

PART 3 · AI辅助SRS生成

LLM生成SRS的能力（2025研究）

能力维度	表现	局限
生成综合SRS	可包含功能和非功能需求	完整性依赖Prompt质量
格式规范化	可按IEEE 830模板输出	编号和交叉引用容易出错
对入门级工程师	生产力提升显著	对资深工程师提升有限
文档审查	可检查完整性、清晰度、正确性	对隐式需求检测不足

关键发现： LLM生成的SRS对初学者帮助最大，但仍需人工审查确保**领域特定需求**的完整覆盖

来源： arxiv.org/pdf/2404.17842, arxiv.org/html/2507.19113v1

课堂演示 – LLM生成SRS片段

Prompt 示例

你是一名需求工程师。请根据以下电梯系统用例描述，按照IEEE 830标准格式生成SRS文档片段，包括：

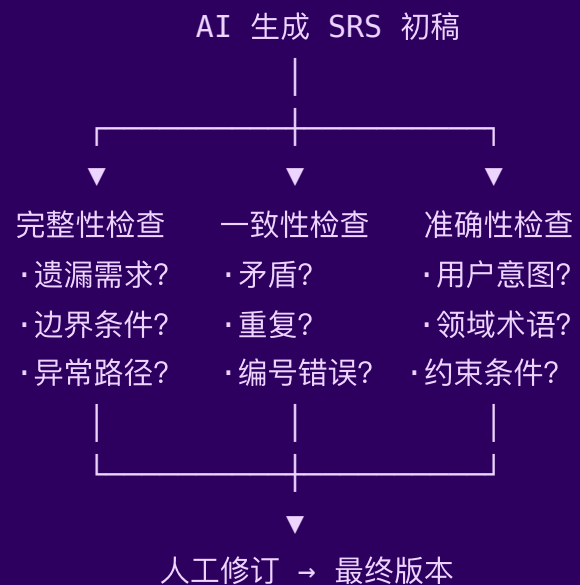
1. 功能需求列表（编号为FR-xxx）
2. 每条需求包含：刺激、响应、相关功能需求ID
3. 非功能需求（性能、安全、可用性各至少1条）

[用例描述粘贴...]

对比维度

对比项	AI生成版	教材标准版
需求覆盖率	?	电梯系统规格说明1-4
编号规范性	?	标准
非功能需求	?	标准

AI生成SRS的审查要点



核心原则： AI生成初稿，人工负责审查和签字

PART 4 · 需求验证

需求验证的标准 [SWEBOK 5.1]

验证标准	检查内容	常见问题
正确性	每条需求准确反映用户意图	误解用户需求、遗漏约束
完整性	需求集覆盖所有场景	遗漏异常流程、边界条件
一致性	需求之间无矛盾	功能冲突、数据定义不一致
可读性	文档易于理解	术语不统一、表述含糊
可修改性	文档易于维护和更新	耦合过紧、缺乏模块化

验证的目标：确保写对了 (Validation)
而非确保写得规范 (Verification)

验证的方法 [SWEBOK 5.1, 5.3]

1. 同级评审 (Peer Review)

- 最常用的验证方法
- 多人交叉审查，发现遗漏和歧义
- 形式：走查 (Walkthrough)、审查会议 (Inspection)

2. 原型 (Prototyping)

- 让用户看到可交互的初步产品
- 快速验证需求理解是否正确
- 特别适合UI密集型系统

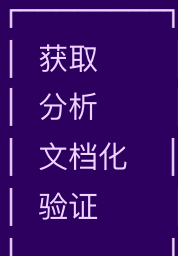
3. 模拟 (Simulation)

- 用模型模拟系统行为
- 适合复杂业务逻辑的验证
- 可发现流程中的逻辑错误

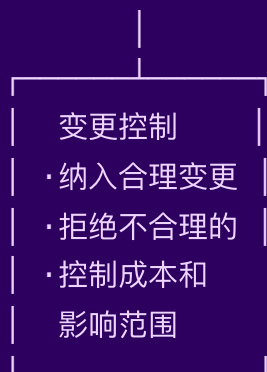
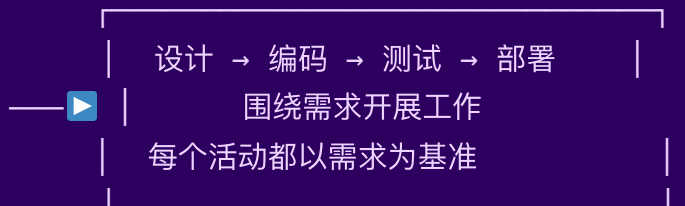
需求管理 [SWEBOK 6.2]

保证需求作用的持续、稳定和有效发挥

需求开发

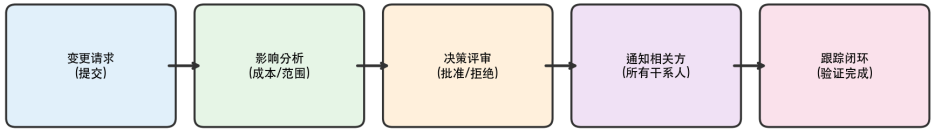


后续活动



需求变更控制 [SWEBOK 6.2]

变更控制流程



瀑布 vs 敏捷中的变更管理

维度	瀑布/计划驱动	敏捷
变更机制	正式的CCB	Backlog 优先级调整
变更时机	需求阶段后正式流程	每个 Sprint 都可接纳
变更成本	越晚越贵 (10x-100x 法则)	短迭代降低变更成本
核心原则	基线管理 + 可追溯	拥抱变化 + 持续交付价值

需求清洗与范围匹配 [SWEBOK 6.2]

需求清洗 (Scrubbing)

- 找到满足利益相关人需要的**最小需求集**
- 砍掉越界/低ROI/不重要的需求

范围匹配 (Scope Matching)

- 确保需求范围不超出**成本、进度和人员约束**
- 超出时需协商裁剪

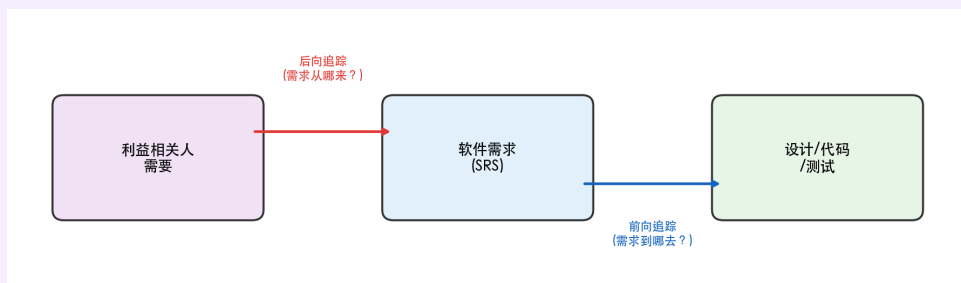
变更管理的核心：纳入合理变更，拒绝不合理变更，控制成本和影响范围

需求可追踪性 (Requirements Tracing) [SWEBOK 7.3]

什么是需求追踪?

记录需求与其他工作产品之间的**对应关系**，确保每条需求都有来源、有实现、有验证。

前向追踪 vs 后向追踪



前向追踪 (Forward)

- 需求 → 设计 → 代码 → 测试
- 确保每条需求都被实现和验证

后向追踪 (Backward)

- 代码 → 设计 → 需求
- 确保每段代码都有需求依据

需求追踪矩阵与核心用途 [SWEBOK 7.3]

需求追踪矩阵 (Traceability Matrix)

需求ID	来源	设计模块	代码模块	测试用例
REQ-001	客户访谈#3	M-Auth	auth.py	TC-001, TC-002
REQ-002	法规要求	M-Security	crypto.py	TC-010
REQ-003	用户故事#7	M-Cart	cart.py	TC-020

追踪的两个核心用途

1. **一致性审计**：每条需求是否都有设计元素来实现？每个设计元素是否都有需求依据？
2. **变更影响分析**：如果 REQ-001 变更 → 追踪矩阵立刻告诉你受影响的设计、代码和测试

需求的附加属性 [SWEBOK 4.5]

除了需求本身的陈述，还应记录哪些元数据？

属性	说明	示例
标识 (ID/Tag)	支持追踪	REQ-001
描述	需求的详细说明	系统应在 3 秒内返回查询结果
理由 (Rationale)	为什么这条需求重要	用户调研显示超过 3 秒会流失 60%
来源 (Source)	谁提出的	产品经理 / 法规 / 用户访谈
优先级 (Priority)	实现顺序	Must-have / Should-have / Nice-to-have

需求的附加属性（续）[SWEBOK 4.5]

属性	说明	示例
稳定性（Stability）	变更可能性	高 / 中 / 低
依赖（Dependencies）	与其他需求的关系	依赖 REQ-003（用户认证）
冲突（Conflicts）	与哪些需求存在矛盾	与 REQ-005（匿名访问）冲突
验收标准	如何验证满足	Given-When-Then 场景
变更历史	修改记录	v1.0 → v1.1 增加了超时限制

这些属性让需求从"一句话"变成可管理、可追踪、可评估的工程产物。
在 AI 时代，这些结构化属性也是 LLM 理解和处理需求的重要输入。

需求的迭代性与稳定性 [SWEBOK 7.1, 7.4]

需求过程的迭代本质

- 需求不仅有广度，还有深度
- 实际项目中，需求工作不可能一次完成
- 有时扩展广度（发现新需求），有时增加深度（细化已有需求）

需求的稳定性频谱

稳定性	特征	示例
非常稳定	软件生命周期内几乎不变	银行计息功能
较稳定	生命周期内可能变化，开发期间不变	账户类型支持
不稳定	开发期间可能多次变化	UI 布局、报表格式
高度不稳定	频繁变化	合规性要求（法规频繁更新）

识别需求的稳定性有助于：为高稳定需求设计不变的核心架构，为低稳定需求设计易于变更的扩展点

PART 5 · AI辅助需求歧义检测

需求歧义检测与LLM (ICSME 2025) [SWEBOK 3.1]

自然语言需求中的歧义类型

歧义类型	描述	示例
词汇歧义	模糊、不精确的词语	"快速"、"方便"、"及时"
句法歧义	句子结构导致多种理解	"用户和管理员可以删除商品" → 谁可以？
语用歧义	需要领域知识才能判断	"系统应支持标准协议" → 哪个标准？

LLM的表现

- 10-shot Prompt比 0-shot 平均提升 20.2%
- 词汇歧义和句法歧义检测较好
- 语用歧义（需要领域知识）表现较弱

来源：ICSME 2025 Industry Track

课堂实验 – AI歧义检测对比（15分钟）

准备10条需求（其中5条含歧义） — 第1组

#	需求描述	歧义?
1	系统应允许用户 快速 查询商品	?
2	系统应在用户输入商品ID后显示商品名称、价格和库存	?
3	管理员和收银员 可以修改商品价格	?
4	系统应在 3秒内 返回查询结果	?
5	系统应该是 安全的	?

课堂实验 – AI歧义检测对比（续）

准备10条需求（其中5条含歧义） — 第2组

#	需求描述	歧义?
6	系统应在每笔交易完成后生成收据编号	?
7	系统应方便地支持批量导入	?
8	每个用户账户最多关联5张银行卡	?
9	系统应及时通知管理员库存不足	?
10	系统应在用户登录失败3次后锁定账户30分钟	?

课堂实验 – AI歧义检测步骤

实验流程



讨论要点

- Zero-shot vs Few-shot: **提供示例**对检测效果的影响
- LLM容易漏掉哪类歧义? → 通常是**语用歧义**
- 人类判断的优势: **领域知识和上下文理解**

PART 6 · 需求冲突检测

需求冲突的类型 [SWEBOK 3.4]

冲突类型	描述	示例
矛盾冲突	两条需求直接矛盾	"系统应自动删除过期数据" vs "系统应永久保留所有数据"
资源冲突	需求竞争有限资源	高性能需求 vs 低成本需求
优先级冲突	利益相关者优先级不同	安全性 vs 易用性
时序冲突	操作顺序上的矛盾	先审核后发布 vs 即时发布

冲突不一定是错误 — 有些冲突需要通过协商和权衡来解决，关键是尽早发现它们

AI冲突识别能力

LLM利用语义相似性检测能力发现相似需求 → 可能指向冲突或冗余

能力边界

检测场景	AI表现	说明
显式矛盾	较好	两条需求文字明确矛盾
隐式冲突	较弱	需要理解业务上下文
跨文档冲突	几乎无法检测	需要全局视图

AI 检测能力谱：

显式矛盾 ————— 隐式冲突 ————— 跨文档冲突
✓ 较好 △ 有限 × 困难
(文字直接矛盾) (需业务知识) (需全局视图)

课堂实验 – AI冲突检测（10分钟）

给LLM一组包含隐藏冲突的需求列表

ID	需求描述
R1	系统应支持匿名用户浏览商品
R2	系统应记录每个用户的浏览历史用于推荐
R3	用户数据应在账户注销后30天内完全删除
R4	系统应保留完整的交易记录用于审计（至少5年）
R5	系统应在1秒内返回搜索结果
R6	搜索结果应包含实时库存信息（每次查询从数据库读取）
R7	管理员可以随时修改商品价格
R8	已加入购物车的商品价格应保持不变直到下单

课堂实验 – 冲突分析与讨论

隐藏的冲突对

冲突对	冲突类型	分析
R1 & R2	矛盾冲突	匿名用户无法记录浏览历史
R3 & R4	矛盾冲突	30天删除 vs 5年保留
R7 & R8	矛盾冲突	随时修改 vs 价格锁定
R5 & R6	资源冲突	1秒响应 vs 实时数据库查询

课堂实验 – 冲突检测实验步骤

实验步骤

1. 让LLM识别冲突 → 记录AI发现了哪些
2. 对比人工识别结果 → AI遗漏了哪些?
3. 讨论: **为什么遗漏?** → 隐式冲突需要业务上下文

AI擅长发现"文字矛盾", 但业务语义层面的冲突仍需人工判断

PART 7 · 工业界前沿：Spec 驱动开发

文档化、验证与冲突检测是需求工程的传统核心。
但 2025 年的工业界正在经历一场范式转型——
从传统的“规格说明”到“规格驱动开发（SDD）”。

Spec-Driven Development (SDD): 2025 最重要的工程实践转型

核心理念

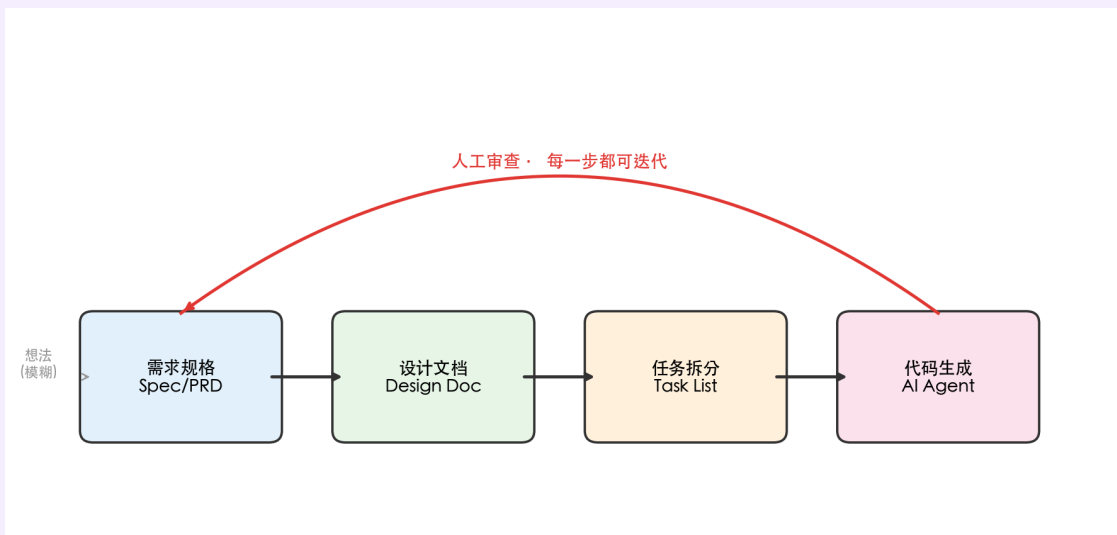
Spec（规格说明）成为软件开发的首要产物，代码是从 Spec 派生的输出。

发展时间线

时间	事件
2025-07	AWS 发布 Kiro ，首次提出 "Spec 即超级 Prompt"
2025-10	GitHub 开源 Spec Kit ，支持 22+ AI Agent 平台
2025-11	Anthropic Claude Code 原生吸收 SDD 工具链
2025-11	ThoughtWorks 将 SDD 列为年度关键工程实践

💡 SDD 不是全新概念——它是需求规格说明在 AI 时代的回归与进化

SDD 核心 workflows: Spec → Design → Tasks → Code



四阶段流水线

1. **Spec** — 写规格说明
2. **Design** — 系统设计
3. **Tasks** — 任务拆解
4. **Code** — AI 生成代码

每一阶段的输出是下一阶段的输入，Spec 质量决定最终代码质量

SDD 核心 workflow：关键洞察

- 每一份文档**依赖于前一份文档**，形成可追踪链条
- 研究发现：**1 次有结构化 Spec 的迭代 $\approx$ 8 次无结构化 Prompt 的迭代精度**
- Spec 不仅是人读的文档，更是 **AI Agent 的输入格式**

SDD 将需求工程的核心价值——**精确的规格说明**——
从"人与人的沟通工具"升级为"人与 AI 的协作协议"

Spec 案例：校园二手交易平台

Feature Spec: 商品搜索与筛选

Problem Statement

学生在二手平台上找不到想要的商品，因为没有有效的搜索和筛选功能，导致用户流失率高达 40%。

Proposed Solution

提供关键词搜索 + 多维度筛选（价格、分类、成色、校区），并按相关度和时间排序展示结果。

User Stories

- As a 买家, I want to 按关键词搜索商品, So that 快速找到目标商品
- As a 买家, I want to 按价格区间筛选, So that 只看预算内的商品

Spec 案例（续）：验收标准与边界

Acceptance Criteria (Given-When-Then)

- Given 数据库中有 500 件商品
When 买家输入"MacBook"并点击搜索
Then 结果在 2 秒内返回, 且包含所有标题含"MacBook"的商品
- Given 买家设置筛选条件: 价格 500-2000, 成色"9成新"
When 点击"应用筛选"
Then 结果只展示满足所有条件的商品

Out of Scope (明确不做)

- 不实现 AI 推荐算法 (v2 再考虑)
- 不支持图片内容搜索
- 不支持跨校区搜索

Open Questions (尚未决定)

- 搜索结果是否需要分页? 每页多少条?
- 是否需要搜索历史记录功能?

💡 注意 Out of Scope 和 Open Questions 的价值:
它们告诉 AI "不要猜测", 大幅减少 AI 的自由发挥空间。

如何写好一份 AI 时代的 Spec?

来源: Addy Osmani (Google Chrome 团队) · O'Reilly 2025

好 Spec 的结构 (6 大要素)

要素	说明
Commands	AI 可执行的具体指令
Testing	验收标准和测试计划
Project Structure	代码组织和目录约定
Code Style	编码规范和约束
Git Workflow	分支和提交规范
Boundaries	AI 行为的边界约束

好 Spec 的边界约束

三层边界系统 (Three-Tier Boundary System)

层级	含义	示例
Always Do	AI 不需要询问即可执行	遵循编码规范、运行测试
Ask First	需要人工确认后执行	删除文件、修改API接口
Never Do	绝对禁止	修改生产数据库、跳过安全检查

边界系统让 AI 知道什么可以自主决定，什么必须请示人类——这正是需求工程中"约束"概念的 AI 时代表达

GitHub Spec Kit: 开源 SDD 工具包

核心能力

- **Feature Spec 模板**: 明确 WHAT (用户需要什么) 和 WHY (为什么), 避免 HOW
- 支持 Claude Code、GitHub Copilot、Gemini CLI 等 22+ AI Agent
- 模板本质上是引导 LLM 行为的结构化 Prompt

GitHub Spec Kit: Feature Spec 模板

模板关键字段

Problem Statement

[用户面临的问题是什么?]

Proposed Solution

[高层次的解决方向, 不涉及技术实现]

User Stories

[As a... I want... So that...]

Acceptance Criteria

[Given-When-Then 格式]

Out of Scope

[明确不做什么—减少 AI 的"自由发挥"]

Open Questions

[尚未决定的问题—让 AI 知道不要猜测]

来源: github.com/github/spec-kit

Claude Code 中的 Spec 驱动开发（一）

Anthropic 团队内部实践

机制	作用	对应需求工程概念
CLAUDE.md	项目"宪法", 定义全局约束	非功能需求 + 项目约束
Subagents	并行处理独立任务	任务分解
Tasks	带依赖关系的任务编排	WBS / Sprint Backlog
Hooks	事件触发自动化动作	验证与质量门禁

Claude Code 将需求工程的核心概念嵌入到了 AI 编程工具的工作流中。

Claude Code 中的 Spec 驱动开发（二）

关键最佳实践

PRD（产品需求文档）是 Claude Code 最重要的输入。

模糊的需求产生模糊的实现。

——Claude Code 官方文档

- 避免长段落包含多个需求——AI 可能遗漏或混合细节
- 需求越详细，AI 输出越准确
- Spec 不仅是人与人的沟通工具，更是人与 AI 的沟通协议

来源：code.claude.com/docs · chatprd.ai

SDD 与传统需求工程：换了名字的老朋友

传统需求工程概念	SDD 对应概念	变化
需求规格说明书 (SRS)	Feature Spec / PRD	格式更轻量，面向 AI 可读
需求层次 (业务→用户→系统)	Problem → Solution → Tasks	结构相同，命名不同
需求验证 (评审)	Spec Review + Conformance Suite	自动化程度提高

SDD 与传统需求工程（续）

传统需求工程概念	SDD 对应概念	变化
变更管理	Spec Drift Detection	新挑战：Spec 与代码不同步
用例描述	User Stories + Acceptance Criteria	更轻量但本质一致
IEEE 830 标准	Spec Template (Spec Kit)	从行业标准到社区模板

核心洞察：SDD 证明了一个事实——
AI 越强大，需求规格说明越重要。
这正是你们学习需求工程的最大价值所在。

课堂讨论：SDD 是"瀑布模型的回归"吗？

反方观点 (marmelab.com, 2025)

- SDD 要求先写完整 Spec 再生成代码，与敏捷快速迭代矛盾
- 代码生成不具有确定性，升级维护困难
- **Spec Drift**: Spec 和代码逐渐不一致

正方观点 + 解决方案

- **Micro-Spec**: 小规模 Spec + 快速迭代 = 兼顾质量与敏捷
- SDD 不要求"一次性写完所有 Spec"
- Spec 可以是 Living Document (活文档)，随项目演化

思考题：在你的团队项目中，
你更倾向于"先写完整 Spec 再让 AI 生成代码"，
还是"边写边改，快速迭代"？为什么？

前沿：RE4AI —— 面向 AI 系统的需求工程

当你开发的系统本身就是 AI 时，需求怎么写？

传统系统需求	AI 系统特殊需求	示例
功能需求	数据需求	训练数据须覆盖所有年龄段，标注准确率 > 95%
性能需求	模型性能需求	推理延迟 < 100ms，准确率 > 90%
安全需求	公平性需求	模型对不同性别/种族的预测差异 < 5%
可维护性	可解释性需求	每个预测结果须提供 Top-3 影响因素
可靠性	漂移监控需求	模型精度下降超过 5% 时自动触发重训练

💡 238 篇论文的系统综述表明：RE4AI 是需求工程最大的新兴方向。课程教了"用 AI 辅助 RE"，但未来你更需要"为 AI 系统写需求"。
来源：Springer 2024 · RE4AI Maturity Study

前沿：多 Agent 辩论（MAD）分类需求 [RE 2025]

原理：让多个 AI 互相辩论，提高需求分类准确率

角色	职责
倡导者 Agent	为某个分类结果辩护，提供论据
批评者 Agent	质疑分类结果，指出漏洞
裁判 Agent	综合双方论据，做出最终判定

实验效果（RE 2025 论文）

方法	F1 分数
单 Agent 直接分类	0.726
多 Agent 辩论（MAD）	0.841 (+15.8%)

💡 这与需求评审中的**同级评审**理念一致——多个视角比单一视角更可靠。
AI 世界也在"重新发明"人类早已验证的工程实践。
来源：arxiv.org/abs/2507.05981

PART 8 · 从自然语言到形式化规约（选讲）

从自然语言到形式化规约 [SWEBOK 3.3]

前沿方向：VERIFYAI 项目 (ACL 2025)

- 将非形式化自然语言需求转化为可验证的形式化证明
- 目标：实现需求的自动化验证

挑战与机遇

维度	现状
LLM形式逻辑推理	仍有局限，复杂逻辑易出错
作为"翻译初稿"工具	可行，人工校验后可用
完全自动化验证	尚未成熟

形式化方法是需求验证的终极目标，
LLM正在成为连接自然语言与形式语言的桥梁

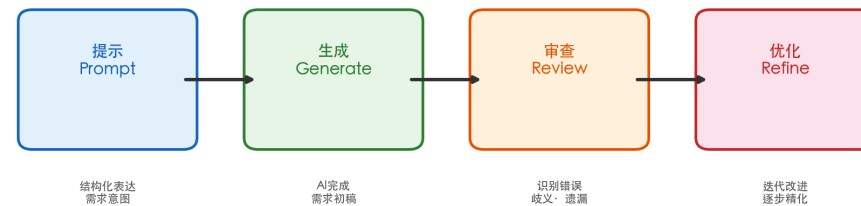
来源：[sciencedirect.com/science/article/pii/S0950584925000369](https://www.sciencedirect.com/science/article/pii/S0950584925000369)

需求模块总结

需求模块总结 – 五节课回顾

- L1 需求工程基础 → 需求本质和层次
- L2 需求获取+AI → 获取方法, AI辅助
- L3 用例建模 → 用例图和用例描述
- L4 分析方法+AI建模 → OO分析, AI生成UML
- L5 文档化+验证 → Spec驱动开发

贯穿主线



需求模块核心能力总结

能力维度	传统要求	AI时代新要求
获取	会访谈、会观察	会用AI辅助获取 + 识别AI幻觉
建模	会画用例图、类图、顺序图	会用AI生成初稿 + 审查修正
文档	会写SRS	会用AI生成 + 人工确保完整性一致性
验证	会评审	会用AI检测歧义冲突 + 识别AI遗漏
核心价值	会写需求	会判断AI写的需求对不对

AI时代的需求工程师：从"需求的编写者"转变为"需求的审判者"

课后作业

以本组项目为背景，完成以下任务：

#	任务	要求
1	编写完整用例文档	至少包含3个用例的详细描述
2	编写SRS片段	至少10条功能需求 + 5条非功能需求
3	用LLM检测歧义和冲突	记录AI的发现
4	人工补充AI遗漏的问题	标注遗漏原因

提交物

- 用例文档 + SRS文档
- AI审查日志（Prompt和AI输出）
- 差异分析报告（AI发现 vs 人工补充）

本节课结束

"需求文档是团队的契约。AI是高效的助手，但签字的永远是工程师。"